Task 3

🎯 Task: JSON → Transform → Parquet + JSON

Scenario

You receive customer orders continuously as JSON files.

Your Spark Streaming job should:

1. Read JSON files as a **stream**
2. Parse the JSON structure
3. Flatten the nested customer information
4. Calculate `total_amount`
5. Keep only useful columns
6. Save the streaming result as:
   - **Parquet**
   - **JSON**
7. Use a **checkpoint** so Spark remembers what it already processed.

---

1. Create the input JSON files

Create this directory:

```
hdfs dfs -mkdir -p /data/streaming/orders
```

Create a local file called:

```
orders1.json
```

Put this inside:


```
{"order_id":"ORD1001","timestamp":"2026-08-
26T10:15:00Z","customer":{"customer_id":501,"name":"John
Doe","email":"john@example.com","address":{"city":"Toronto","postal_code":"M5
V2T6"}},"product":"Laptop","category":"Electronics","quantity":1,"price":1200
.0}
{"order_id":"ORD1002","timestamp":"2026-08-
26T10:20:00Z","customer":{"customer_id":502,"name":"Jane
Smith","email":"jane@example.com","address":{"city":"Vancouver","postal_code"
:"V6B1A1"}},"product":"Mouse","category":"Electronics","quantity":2,"price":2
5.0}
{"order_id":"ORD1003","timestamp":"2026-08-
26T10:25:00Z","customer":{"customer_id":503,"name":"Mike
```

```
Brown","email":"mike@example.com","address":{"city":"Montreal","postal_code":
"H2X1Y4"}},"product":"Keyboard","category":"Electronics","quantity":1,"price"
:75.0}
```

Upload it:

```
hdfs dfs -put orders1.json /data/streaming/orders/
```

---

2. Create another JSON file later

Create:

```
orders2.json
{"order_id":"ORD1004","timestamp":"2026-08-
26T11:00:00Z","customer":{"customer_id":504,"name":"Sarah
Wilson","email":"sarah@example.com","address":{"city":"Calgary","postal_code"
:"T2P1J9"}},"product":"Monitor","category":"Electronics","quantity":2,"price"
:350.0}
{"order_id":"ORD1005","timestamp":"2026-08-
26T11:10:00Z","customer":{"customer_id":505,"name":"David
Lee","email":"david@example.com","address":{"city":"Ottawa","postal_code":"K1
P1J1"}},"product":"Headphones","category":"Audio","quantity":3,"price":80.0}
```

Don't upload `orders2.json` immediately.

We'll use it to simulate a new batch of streaming data.

---

3. Your Spark Structured Streaming code

Run this in **one Jupyter cell**:

```
# ============================================================
# Spark Structured Streaming
# JSON -> Transform -> Parquet + JSON
# ============================================================

from pyspark.sql import SparkSession
from pyspark.sql.functions import *
from pyspark.sql.types import *
from pyspark.sql.window import Window


# ============================================================
# 1. Create Spark Session
# ============================================================

spark = (
```

```
    SparkSession.builder
    .appName("JSON Streaming Orders")
    .getOrCreate()
)

print("Spark version:", spark.version)


# ============================================================
# 2. Define JSON schema
# ============================================================

schema = StructType([
    StructField("order_id", StringType(), True),
    StructField("timestamp", StringType(), True),

    StructField(
        "customer",
        StructType([
            StructField("customer_id", IntegerType(), True),
            StructField("name", StringType(), True),
            StructField("email", StringType(), True),

            StructField(
                "address",
                StructType([
                    StructField("city", StringType(), True),
                    StructField("postal_code", StringType(), True)
                ]),
                True
            )
        ]),
        True
    ),

    StructField("product", StringType(), True),
    StructField("category", StringType(), True),
    StructField("quantity", IntegerType(), True),
    StructField("price", DoubleType(), True)
])


# ============================================================
# 3. Read JSON as a STREAM
# ============================================================

orders_stream = (
    spark.readStream
    .schema(schema)
    .json("hdfs://localhost:9000/data/streaming/orders")
)


# ============================================================
# 4. Transform the streaming DataFrame
# ============================================================
```

```
orders_transformed = (
    orders_stream

    # Convert timestamp from String -> Timestamp
    .withColumn(
        "timestamp",
        to_timestamp("timestamp")
    )

    # Extract nested customer information
    .withColumn(
        "customer_id",
        col("customer.customer_id")
    )

    .withColumn(
        "customer_name",
        col("customer.name")
    )

    .withColumn(
        "customer_email",
        col("customer.email")
    )

    .withColumn(
        "city",
        col("customer.address.city")
    )

    .withColumn(
        "postal_code",
        col("customer.address.postal_code")
    )

    # Calculate total amount
    .withColumn(
        "total_amount",
        col("quantity") * col("price")
    )

    # Select final columns
    .select(
        "order_id",
        "timestamp",
        "customer_id",
        "customer_name",
        "customer_email",
        "city",
        "postal_code",
        "product",
        "category",
        "quantity",
        "price",
        "total_amount"
    )
)
```

```
# ============================================================
# 5. Output paths
# ============================================================

parquet_output = "hdfs://localhost:9000/data/streaming/output/parquet"

json_output = "hdfs://localhost:9000/data/streaming/output/json"

checkpoint_parquet =
"hdfs://localhost:9000/data/streaming/checkpoint/parquet"

checkpoint_json = "hdfs://localhost:9000/data/streaming/checkpoint/json"


# ============================================================
# 6. Write STREAM to Parquet
# ============================================================

parquet_query = (
    orders_transformed.writeStream
    .format("parquet")
    .outputMode("append")
    .option("path", parquet_output)
    .option("checkpointLocation", checkpoint_parquet)
    .trigger(processingTime="10 seconds")
    .start()
)


# ============================================================
# 7. Write STREAM to JSON
# ============================================================

json_query = (
    orders_transformed.writeStream
    .format("json")
    .outputMode("append")
    .option("path", json_output)
    .option("checkpointLocation", checkpoint_json)
    .trigger(processingTime="10 seconds")
    .start()
)


# ============================================================
# 8. Show streaming information
# ============================================================

print("Streaming started!")

print("Parquet query:", parquet_query.id)

print("JSON query:", json_query.id)
```

```
# ============================================================
# 9. Keep Spark Streaming running
# ============================================================

parquet_query.awaitTermination()
```

Important

The code starts **two streaming queries**:

```
                  ┌──> Parquet
JSON files ──> Spark Streaming
                  └──> JSON
```

Each output has its **own checkpoint**.

That's important because each streaming sink maintains its own progress.

---

4. Check that the streaming query is running

While the notebook is running, open another Ubuntu terminal:

```
jps
```

Then check:

```
hdfs dfs -ls -R /data/streaming
```

You should eventually see something similar to:

```
/data/streaming
├── orders
│   ├── orders1.json
│
├── output
│   ├── parquet
│   │   └── part-....
│   │
│   └── json
│       └── part-....
│
└── checkpoint
    ├── parquet
    └── json
```

---

5. Read the Parquet output

Stop the streaming query with:

```
parquet_query.stop()
json_query.stop()
```

Then read the output as a normal batch DataFrame:

```
df_parquet = spark.read.parquet(
    "hdfs://localhost:9000/data/streaming/output/parquet"
)

df_parquet.show(truncate=False)
```

You should get something like:

```
+--------+-------------------+-----------+-------------+------------------+--
-------+-----------+----------+-----------+--------+------+------------+
|order_id|timestamp          |customer_id|customer_name|customer_email
|city     |postal_code|product   |category   |quantity|price |total_amount|
+--------+-------------------+-----------+-------------+------------------+--
-------+-----------+----------+-----------+--------+------+------------+
|ORD1001 |2026-08-26 10:15:00|501        |John Doe     |john@example.com
|Toronto  |M5V2T6     |Laptop   |Electronics|1       |1200.0|1200.0      |
|ORD1002 |2026-08-26 10:20:00|502        |Jane Smith   |jane@example.com
|Vancouver|V6B1A1     |Mouse    |Electronics|2       |25.0  |50.0        |
|ORD1003 |2026-08-26 10:25:00|503        |Mike Brown   |mike@example.com
|Montreal |H2X1Y4     |Keyboard |Electronics|1       |75.0  |75.0        |
+--------+-------------------+-----------+-------------+------------------+--
-------+-----------+----------+-----------+--------+------+------------+
```

---

6. Check the JSON output

```
df_json = spark.read.json(
    "hdfs://localhost:9000/data/streaming/output/json"
)

df_json.show(truncate=False)
```

---

7. Now simulate NEW streaming data 🚀

This is the important part.

Start the streaming code again.

Then, **while Spark is running**, upload the second file:

```
hdfs dfs -put orders2.json /data/streaming/orders/
```

Spark Structured Streaming is watching:

```
/data/streaming/orders/
```

It detects:

```
orders1.json  ← already processed
orders2.json  ← NEW
```

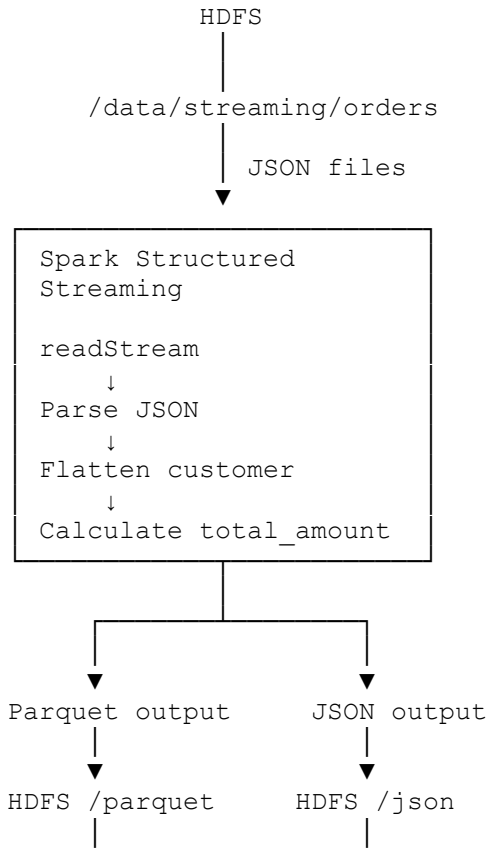
and processes the new file.

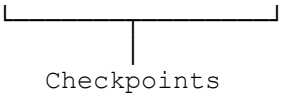
You don't restart Spark.

That's the basic idea of **file-based Structured Streaming**.

---

8. Your final architecture





🧠 What you're practicing

This single project teaches you:

- `readStream`
- JSON schema
- nested JSON
- `StructType`
- `StructField`
- `withColumn`
- `col()`
- `to_timestamp()`
- flattening nested data
- calculated columns
- `writeStream`
- `append` mode
- Parquet sink
- JSON sink
- checkpointing
- `trigger(processingTime=...)`
- `awaitTermination()`
- adding new data while the stream is running

Next challenge: modify this project to read the same JSON stream and produce **three outputs: cleaned Parquet + aggregated sales by category + a console output**, which will introduce you to streaming aggregations and watermarks.